

Benchmarking Landing & Delivery Zone Detection Algorithms: Insights on Open Evaluation Framework.

1st Giridhar Vitta Bukka

Institute of Flight Systems

University of the Bundeswehr Munich

Neubiberg, Germany

giridhar.vb@unibw.de

2nd Peter Stütz

Institute of Flight Systems

University of the Bundeswehr Munich

Neubiberg, Germany

peter.stuetz@unibw.de

Abstract—The rapid proliferation of hub-to-hub and last-mile Unmanned Aerial System (UAS) cargo delivery services, though currently small-scale, underscores the need for reliable, robust, and computationally efficient Landing and Delivery Zone Detection Algorithms (LDZDAs) to ensure safe large-scale deployment. The extensive real-time data generated during UAS missions necessitates efficient algorithms to identify safe zones of interest amidst various hazards. Implementing LDZDAs in varied scenarios demands customized strategies, as factors such as observation distance, terrain complexity, and environmental conditions require diverse problem-solving approaches and modalities. Yet there is a notable lack of public benchmarks for comprehensive evaluation at the architectural and implementation levels. An open-source benchmarking framework, developed collaboratively by UAS researchers, developers, and operators, would standardize acceptable performance and provide regulatory bodies with a reference for state-of-the-art development. Common LDZDA architectures leverage sensors such as Global Navigation Satellite System (GNSS), cameras, LiDAR, spectral, radar, and thermal imaging. This paper proposes a multi-stage evaluation framework to assess LDZDA performance. To begin with this paper proposes three stages as follows. The first stage evaluates algorithm behavior over predefined hazard metric values ranging from unacceptable to acceptable covering also the transition in between. The second stage assesses performance across varying terrain complexities. The third stage shall examine adaptability to adverse environmental conditions, compliance with regulatory requirements, and hardware deployability, offering insights for targeted R&D. As an initial case study, this work focuses on first stage of evaluation and point cloud-based landing zone detection algorithms (PC-LZDs). Three traditional rule based PC-LZD architectures and few sub variants are implemented and benchmarked in the first stage, evaluating success rate, computation time, and resource usage. Discussion on other state of the art PC-LZD methods including Deep Learning based and compatibility with the proposed Monte-Carlo based evaluation framework for benchmarking is also provided. Preliminary insights into the second stage evaluation is also provided. This study along side highlighting the need for an open standard framework for benchmarking LDZDAs aims to guide the selection and design of LDZDAs for specific use cases, enhancing safe and efficient UAS cargo delivery.

Index Terms—cargo, landing, delivery, site, zone, area, detection, evaluation, UAS, UAV, gesture, drone, visual communication, Monte Carlo, AirSim, Benchmarking, Evaluation, Performance, Coarse-to-Fine.

I. INTRODUCTION

Autonomous UAS landing and delivery, including fixed-wing and VTOL systems, face challenges at unprepared sites due to complex terrain, adverse weather, and dynamic obstacles. Multi-modal Landing & Delivery Zone Detection Algorithms (LDZDAs) are crucial for real-world applications like cargo delivery, except in controlled environments. Sensor selection, based on observation distance, terrain complexity, and environmental conditions, is key for effective perception. For example, Point-cloud-based Landing Zone Detection (PC-LZD) algorithms using LiDAR provide high-resolution terrain and geometric data, while image-based LDZDAs offer valuable visual information across distances. Tailoring these algorithms to specific scenarios, such as urban settings, disaster response, planetary exploration, or human-drone interactions, is essential for operations without pre-mapped landing sites [1]–[3].

A. Context

Commercial drone delivery services, such as those by Google’s Wing, Zipline, Matternet, Flytrex, Auterion, and Phoenix-Wings, have progressed from trials to operational services in select regions, driven by advancements in autonomous flight, regulatory approvals, and demand for fast, sustainable delivery. However, scaling is limited by manual site assessment using satellite imagery, GIS, and DEMs, which risks becoming a bottleneck [4]. In the U.S., the FAA’s UAS Integration Pilot Program and BEYOND initiatives, and in Europe, EASA’s efforts, foster safe UAS integration through collaborative research on Detect and Avoid, communications, human factors, safety, and certification [5]. As small-payload UAS become reliable, their use in larger cargo delivery grows, requiring standardized autonomous landing and delivery capabilities for routine and emergency scenarios. However, there

is a lack of public benchmarks for evaluating these systems at architectural and implementation levels. While academic research often tests algorithms on public datasets to assess robustness, synthetic data via Monte Carlo simulations offers deeper insights into edge-case performance, which the proposed benchmarking framework aims to address.

B. Objectives

Safe landing and delivery require identifying suitable environmental interaction zones. This paper focuses on point-cloud-based Landing Zone Detection (PC-LZD) algorithms, building on prior work [7] that introduced a three-stage evaluation strategy for robustness, accuracy, and efficiency. We propose a refined multi-stage evaluation framework for LDZDAs, initially comprising three stages, to enhance real-world applicability. The first stage uses Monte Carlo simulations, inspired by [6], to assess PC-LZD performance under hazard metrics (e.g., minimum landing zone radius, maximum terrain slope, point density, local relief, and roughness) that quantify site suitability [7]. Key metrics include success rate (identifying viable landing zones), computation time, and resource utilization. Three rule based PC-LZD architectures are implemented: a KD-tree-based inflating circles algorithm [6], a sliding window sequential processing algorithm with three variants (non-overlapping, overlapping, and KD-tree-enhanced non-overlapping) [1], [6], [10]–[14], and a region-growing segmentation-based algorithm [15]. These leverage open-source C++ libraries, including the Point Cloud Library (PCL) [16], Computational Geometry Algorithms Library (CGAL) [17], and Open3D [18], for efficient point-cloud processing. A modular C++ library for Monte Carlo simulations enables performance analysis across hazard metric ranges and supports integration of new algorithms and metrics. To foster collaboration and regulatory alignment, the algorithms, simulation framework, and LZD components are available at <https://github.com/EXPX3/Drone-Delivery-Landing-Zone-Detection> for feedback and improvement. LDZDAs primarily comprise of processing input data types like images and pointclouds. While currently applied to PC-LZD algorithms in this paper, the suitability of such framework to image-based LDZDAs seems plausible as the image characteristics like image gradient (surface smoothness), pixel intensity variance (roughness), edge detection (obstacle offset), pixel intensity variations (relief) etc that can be generated in a controlled fashion and measured via image processing exist. The second stage evaluates performance across varied terrain complexities, and the third stage examines adaptability to adverse environmental conditions, regulatory compliance, and hardware deployability, guiding targeted research.

This work benchmarks the three PC-LZD architectures in the first stage, analyzing success rate, computation time, and resource usage, with preliminary insights into the second stage. A standardized LDZDA evaluation framework is essential for accelerating regulatory approval and advancing UAS applications. Post-simulation analysis revealed algorithm anomalies,

highlighted weaknesses, and identified conditions where specific architectures excel. Two key challenges emerged: extracting boundaries of complex-shaped flat areas and ensuring landing zones are suitable for vertical descent, as candidates may lie beneath obstacles (e.g., trees or overhangs). To address these, a C++ module for boundary extraction, circular shape fitting, and vertical collision checking was developed, with its benefits and limitations discussed. Community-validated components like these could be integrated into the open-source library alongside the evaluation framework.

C. Contributions and Paper Organization

The main contributions of this work are:

- A Monte Carlo simulation-based evaluation framework for benchmarking LDZDAs and how it can serve as a tool to tune the algorithms for specific use cases.
- Implementation of three PC-LZD architectures: KD-tree-based inflating circles, a sliding window family with three variants (non-overlapping, overlapping, and KD-tree-enhanced non-overlapping), and region-growing segmentation-based.
- A novel KD-tree-enhanced sliding window variant, benchmarked for efficiency.
- Comprehensive evaluation of algorithm success rate, computation time, and resource utilization, with statistical analysis.
- Discussion of challenges in achieving a 100% success rate.

The paper is organized as follows: Chapter II reviews related work, emphasizing LDZDAs algorithm challenges and the need for standardized evaluation. Chapter III describes the LZD algorithms, implementation details, and preliminary results. Chapter IV details the experimental setup, including datasets, hardware, and software configurations. Chapter V outlines the current three-stage evaluation framework, focusing on Monte Carlo simulations and terrain complexity analysis. Chapter V also presents evaluation results, including performance metrics and algorithm comparisons. Chapter VI summarizes findings and proposes future research directions.

II. RELATED WORK

LDZDAs for UAS are designed to identify safe landing and delivery zones, with algorithm choice depending on operational context. Core processes typically include data filtering, downsampling, detection, and segmentation, tailored to environmental demands—urban operations may prioritize flat rooftops, while rural settings focus on clearings in forested areas. For a quick introduction on the variety of sensors that potentially can be used for LDZDAs and an overview on point cloud based LZDAs please refer to [7]. Popularly used sensors include LiDAR and cameras given their ability to provide high-resolution spatial and visual data respectively.

Researchers have developed diverse Landing Zone Detection Algorithms (LZDA) approaches, each with distinct strengths. Spatial discretization is commonly used to manage 3D point-cloud data. For instance, [1], [10], [19] adopt a

3D grid map, dividing space into voxels populated with sampled environmental points, while [20], [21] employ a 2D grid capturing elevation data per cell. Extracting planar surfaces, critical for landing zone identification, relies on fitting techniques such as RANSAC [1], [10], PROSAC [13], [15], Least Squares [20], and Least Median of Squares [22], [23]. PROSAC, as noted in [15], outperforms RANSAC by using prioritized sampling to select high-quality data points, reducing randomness and improving efficiency. Least Squares enables real-time processing [20], while Least Median of Squares offers robustness and speed over RANSAC [23]. For time-critical applications, [24] favors an Average 3D Gradients approach over Principal Component Analysis (PCA) for its simplicity and execution speed [7]. These traditional rule based methods use heuristic rules that need to be tuned for specific use cases but are generally fast and efficient. In [25], Deep Learning (DL) method were applied to detect LZs, point clouds were projected and converted to 2D range images and transformer based models were trained. In [26], DL method was used for semantic segmentation, in [27] and [28] to directly process point clouds to identify LZs. The proposed Monte Carlo simulation-based evaluation framework for benchmarking in this paper can be used to benchmark these DL based PC-LZD algorithms as well.

Open-source implementations provide practical insights. The PX4 autopilot's `safe_landing_planner` module processes point-cloud data from a downward-facing sensor, organizing it into a 2D grid based on XY coordinates and calculating the mean and standard deviation of Z-coordinates per cell to identify flat surfaces [29]. Deep learning-based solutions, such as OpenLander, leverage Deep Neural Networks for landing zone detection [30], offering an alternative to traditional methods.

Despite the variety of proposed LDZDAs, significant gaps persist. Most studies do not release implementation code, forcing researchers to reimplement algorithms or rely on unverified claims. Additionally, the absence of publicly available benchmarking results and standardized evaluation frameworks hinders objective comparisons of real-world performance. Such a framework would facilitate independent validation, lower barriers for new researchers, and promote safer, industry-wide deployment standards for UAS landing and delivery operations.

III. ALGORITHMS SELECTED AND IMPLEMENTED FOR BENCHMARKING

A. Overview of Selected Algorithms

This section describes three LZD architectures inspired by the literature: Sliding window sequential processing [1], [10]–[14], Kd-tree based inflating circles [6] and Region-growing segmentation [15] based. The implementations in this open sourced work may differ from the original studies due to the unavailability of exact details. The first and third architectures initially identify a set of flat regions filtered by predefined flatness-based hazard metric threshold, rather than directly providing LZs candidate coordinates for UAS final descent and

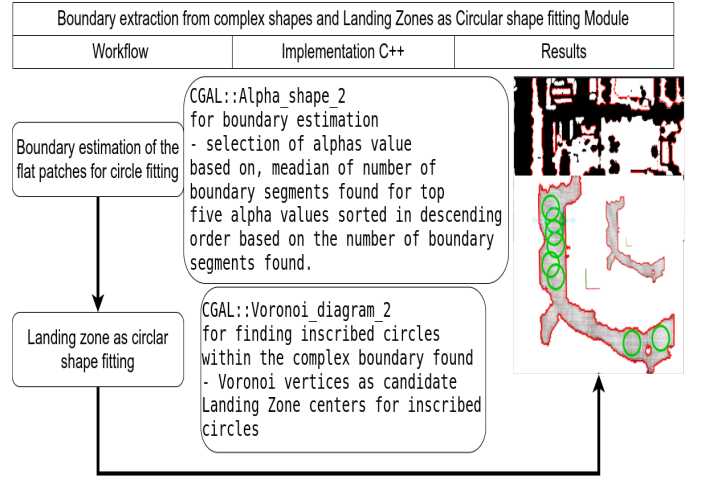


Fig. 1: Workflow on the left with specific C++ libraries used for boundary extraction and circle fitting in the middle, and results of complex shape boundary extraction and circle fitting on the right. In the results section, the third coloumn, upper part shows complexity in shapes that need to be extracted, below part shows an example of a successful boundary extraction and circle fitting.

landing. For practical applications, UASs require a reference coordinate as the next waypoint for landing, a requirement common to many LZDAs that first produce flat regions. In the below four sections, first the common components of LZDAs are described, followed by the three selected architectures.

1) Common Components for LZDAs:

a) *Boundary Extraction of Complex Shapes:* As a secondary step, a practical module for boundary extraction and circular shape fitting was implemented. Figure 1 illustrates the workflow, implementation details in C++, and results. The top-right region of the figure demonstrates the complexity of shapes, which varies with terrain complexity and the LZDA used. As shown in Figure 1, the implemented module successfully extracts boundary segments of complex shapes. The approach utilizes `CGAL::Alpha_shape_2` [17], where the choice of the alpha parameter is critical for achieving an accurate boundary fit. Typically, this requires a trial-and-error approach with visual inspection to determine the optimal alpha value. To enable real-time, autonomous operation without human intervention, a community-suggested method was adopted: iteratively running boundary extraction with different alpha values over a preset range, storing the number of boundary segments for each value, and selecting the value with the maximum number of segments. However, experiments revealed that this approach is not always effective. A higher number of boundary segments may indicate an overly complex shape, causing circle fitting to fail, while too few segments may result in a loosely estimated boundary, where fitted circles may not lie within a valid flat region. To address this, the median value of the sorted alpha values, based on the number of segments, was used. While this does not guarantee a 100%

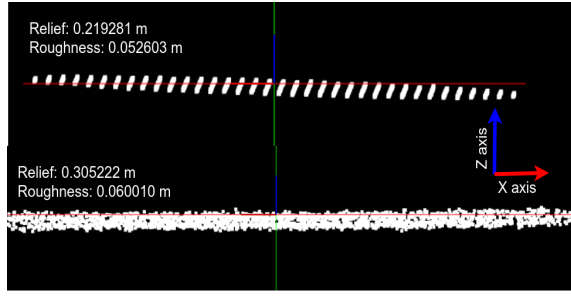


Fig. 2: Detected circular candidate Landing Zone's (LZ) terrain distribution projected on the XZ plane, where Z is along the normal of the candidate LZ. Relief and roughness are calculated over this distribution. The upper distribution is for a controlled synthetically generated point cloud, hence a uniform separation between the points, and the lower distribution is from a real-world point cloud from the OpenTopography dataset [31].

success rate, it provides a robust starting point.

b) *Hazard Metrics Evaluation and Vertical Collision Check*: In [7], the authors list commonly used hazard metrics for selecting ideal LZ candidates. This work adopts the same radius, slope, point density, relief and roughness as hazard metrics as a starting point. These metric thresholds except point density are calculated based on the selected UAS dimensions and landing gear. Although flat regions are detected, boundaries extracted, and circles fitted, not all candidates are suitable for vertical landing due to potential obstacles, such as trees or overhangs, above the detected LZ candidates. Relying on the UAS's local trajectory planner for final descent under obstacles is risky and impractical for larger UASs. To address this, a kd-tree-based module was implemented to check for vertical collisions. The module ensures no LiDAR-sampled points from obstacles exist directly above the candidate LZ. It uses the Min_Z and Max_Z values of the point cloud, starting at Min_Z as the z-coordinate, with x, y coordinates at the center of the candidate circle and a search radius equal to the candidate circle radius. The module incrementally collects neighbors up to Max_Z , effectively capturing an approximately cylindrical volume of space that includes the candidate LZ's terrain distribution at ground level and any obstacles above. Figure 2 shows the projection of the candidate LZ terrain distribution on the XZ plane. Additionally, points are projected onto the XY plane to calculate point density. Based on preset threshold values for relief, roughness, and data confidence, the algorithm filters out unsuitable candidates.

2) Sliding Window Sequential Processing:

a) *Algorithm 1*: This deterministic approach ensures a landing zone is found if one exists, sequentially processing grid cells until all are covered. Figure 4 (left) illustrates a square window sliding over a grid with or without overlapping. Compared to random approaches, this method may delay finding a landing zone in larger areas. However, using computationally efficient steps, it can identify all possible

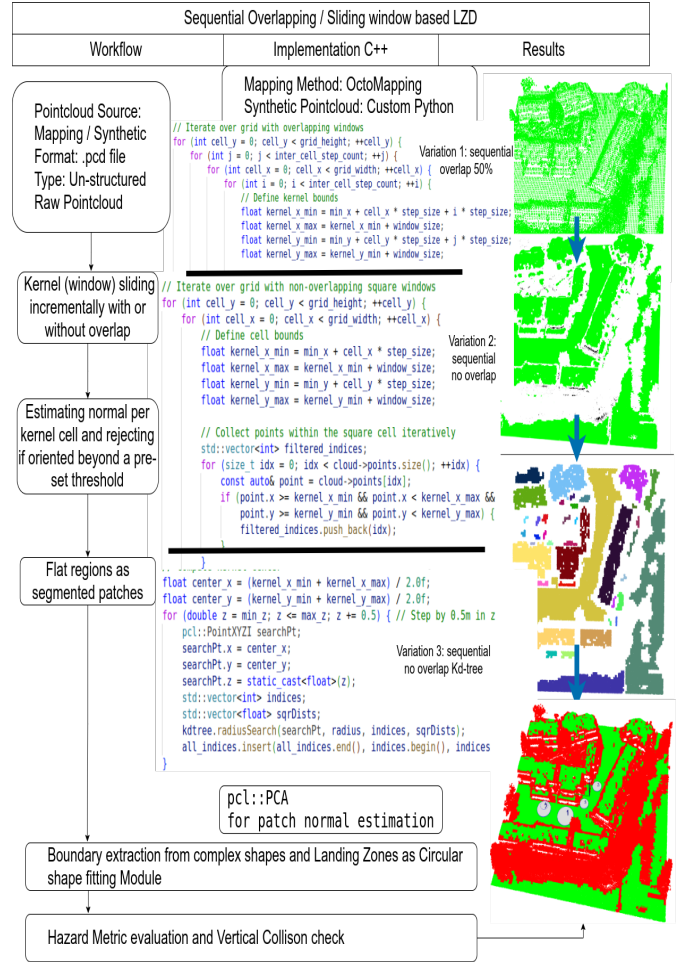


Fig. 3: General workflow of the sliding window LZDA on the left, code snippets of its variations in the middle, and the start-to-end result of the LZDA on the right.

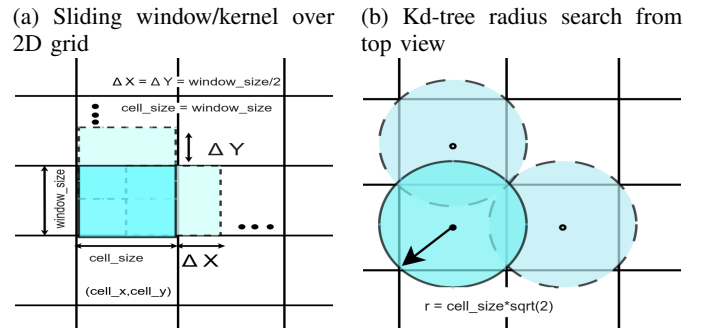


Fig. 4: Sliding window in sequential/overlapping order on the left, where points are iteratively checked for window boundary conditions and collected, and kd-tree radius search on the right, scooping the cell volume from a top view to efficiently collect points.

solutions. By adjusting the Euclidean distance between the centers of consecutive sliding windows along the x and y axes, the algorithm can process grid cells in an overlapping

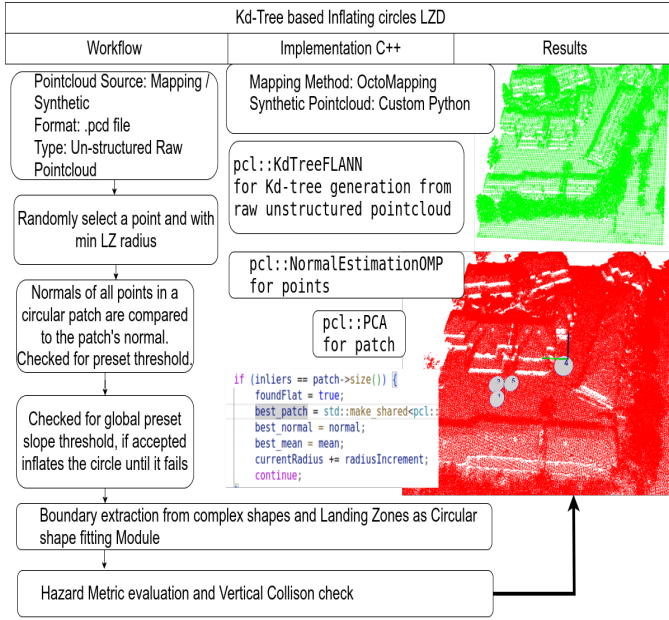


Fig. 5: Pipeline of the Kd-tree inflating circles algorithm, including implementation details and start-to-end results.

manner. Three variations of sliding window-based LZDAs were implemented:

- 1) Variation 1: Sequential processing with 50% overlapping. Figure 3 shows this variation. The 50% overlap ensures that cell edge anomalies are not missed, but it is computationally expensive.
- 2) Variation 2: Sequential processing without overlapping. Figure 3 illustrates this variation, which is computationally lighter and faster but risks missing edge anomalies due to iterative point retrieval within the sliding window.
- 3) Variation 3: Sequential processing without overlapping, using kd-tree radius search to retrieve points within the sliding window. Figure 3 and the right side of Figure 4 show this variation, where kd-tree radius search collects neighbors from a spherical volume between Min_Z and Max_Z , centered at the window (kernel) center, effectively scooping an approximately cylindrical volume for each cell.

As described in Section III-A1, the identified flat regions are processed by the boundary extraction and circle fitting module, followed by the hazard metrics evaluation and vertical collision check module. For a detailed step-by-step workflow, refer to the flowchart in Figure 7.

3) Random Processing Order:

a) *Algorithm 2: Inflating Circles/Random Sampling-Based Landing Zone Detection Using Kd-Tree:* This probabilistic approach does not guarantee a solution, even if one exists, due to random seed point selection within the point cloud. However, for VTOL-UAS applications with bounded search regions, it typically identifies LZ candidates. A kd-tree-based radius search collects points within the seed point's radius. In the literature, octree-based radius searches are also

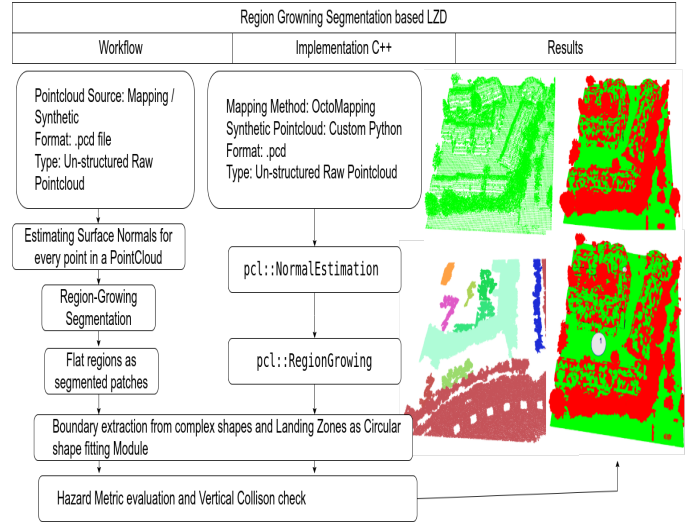


Fig. 6: Workflow and implementation details of the region-growing segmentation LZDA.

used. Figure 5 outlines the workflow, implementation details, and results of this LZDA. For a detailed step-by-step workflow, refer to the flowchart in Figure 7.

4) *Prioritized Processing Order:* In Region-Growing Segmentation, processing prioritizes the point with the minimum curvature value to initiate segmentation.

a) *Algorithm 3: Region-Growing Segmentation-Based Landing Zone Detection:* This deterministic approach guarantees finding a landing zone if one exists, prioritizing the starting point for region-growing based on the curvature value at that point. Figure 6 outlines the workflow, implementation details, and results of this LZDA. For a detailed step-by-step workflow, refer to the flowchart in Figure 7.

5) *Coarse-to-Fine-Based Landing Zone Detection:* The fundamental LZDAs discussed above can be run with coarse settings (e.g., higher downsampling filter) or without downsampling, as done in this work. The coarse-to-fine approach involves two rounds: the first with coarser parameters (e.g., larger voxel size for downsampling and cell size for 2D grid data structuring), followed by finer settings applied to the results from the coarse round. This balances speed and accuracy, making it suitable for real-time applications. In this work, only fundamental methods are evaluated.

B. Implementation Details and Optimizations

Standard libraries, including PCL, Open3D, CGAL, and Eigen, were used to leverage efficient and documented implementations of subcomponents wherever possible.

IV. EXPERIMENTAL SETUP

A. Point Cloud Sources

1) *Synthetic Point Cloud:* Synthetic point cloud datasets were generated using a custom simulation framework implemented for first stage of evaluation, as illustrated in Figure 10. These point clouds are customizable based on hazard metrics

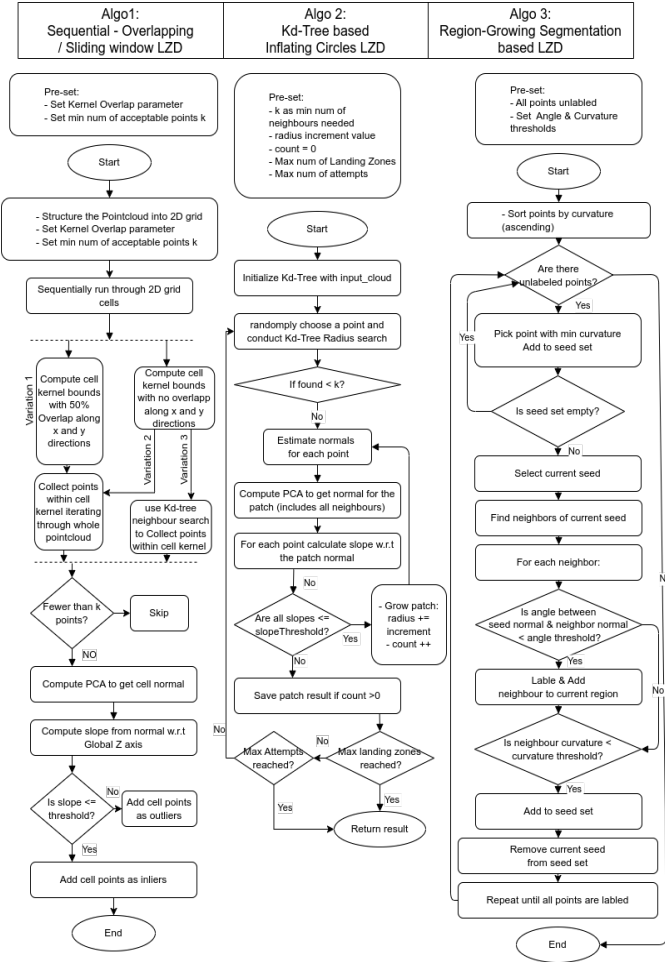


Fig. 7: Detailed flowcharts of all LZDAs benchmarked in this paper.

and area dimensions by the PointCloudGenerator component, refer Figure 9.

2) *Simulated Point Cloud Datasets from Realistic Environments*: A simulation framework integrating AirSim, ROS2, and Python was developed to generate realistic point cloud datasets for UAS navigation and topographic data collection. The setup involves a UAS navigating to a designated coordinate, performing a square pattern flight to capture high-resolution point clouds using a ROS2-simulated LiDAR topic in AirSim, configured with a 100 kHz pulse rate and a 360° horizontal/45° vertical FoV. Three environments were evaluated: a simple blocks environment, a moderate city environment, and a complex post-disaster terrain created in Unreal Engine, as referenced in prior work [32]. Visuals of these environments and point clouds are shown in Figure 14. The study focuses on point cloud-based Landing Zone Detection Algorithms (LZDAs), noting limitations in representing real-world features like water or non-rigid surfaces, underscoring the need for multi-modal LZDA approaches.

While AirSim was used, its limitations in simulating complex dynamic environments and lack of ongoing support

Three-Stage Evaluation Procedure for LZD algorithms		
Stage 1:	Stage2:	Stage3:
Monte-Carlo simulations: To study Robustness, Precision, Accuracy, Computation Time and Resources for LZD algorithms	Real-world Terrain & Scenario complexity Simulation	Real-world / Real-Time Adoption challenges
Robustness: - LZD Success rate - Number of LZs Precision & Accuracy: - Behaviour of LZDAs on parameter variations, Filters etc. Time & Resource analysis: - as feedback for LZDA optimization - real time/world hardware suitability	- Abundance of Opensource Datasets (ex:Open Topography) - Synthetic Data generation for simple but specific terrains - Modern Hyper realistic Simulation tools like IsaacSim, AirSim(Deprecated) provide feasibility to simulate almost any application specific environment and scenario	Adoption of LZDAs demands: - Adverse Environment Contions handling - Regulatory certification requirements - R&D Hardware & Software (Sensors and Data processing units)

Fig. 8: Three-stage evaluation framework for LZDAs.

were noted. IsaacSim, though not detailed in this study, shows promise for generating complex dynamic environments and point clouds, supported by a growing community. Both platforms allow environmental noise simulation (e.g., rain, fog, dust). Future work will shift to IsaacSim for enhanced simulation capabilities.

3) *Open-Sourced Real-World LiDAR Datasets*: To complement the simulated data, standard datasets from OpenTopography [31], representing three diverse terrain types—simple, moderate city, and complex hilly terrain—were collected and tested. These publicly available datasets provide realistic representations of various topographies.

B. Hardware and Software Configuration

The algorithms were executed on a Dell Precision 5760 mobile workstation. Exact details refer to equipped with an Intel® Xeon® W-11955M processor (8 cores, 16 threads, 2.60 GHz base, up to 5.00 GHz turbo, 24 MB L3 cache), 64 GB DDR4 RAM (2 x 32 GB, 3200 MHz), and a dual-GPU configuration featuring an NVIDIA® RTX A3000 Mobile (6 GB GDDR6, 5,120 CUDA cores) for high-performance computing tasks and an integrated Intel® UHD Graphics (Tiger Lake-H GT1) for lightweight rendering. The system ran Ubuntu 22.04 LTS, leveraging NVIDIA driver version 535.104.05 and CUDA 12.2 for GPU-accelerated computations, providing robust processing capabilities for large-scale datasets and parallelized algorithmic workloads.

V. PROPOSED THREE-STAGE EVALUATION FRAMEWORK OVERVIEW, RESULTS AND DISCUSSION

A. Overview of the Evaluation Framework

The benefits of adopting a standardized evaluation framework, as outlined in Figure 8, are multifaceted for UAS deployers, developers, and regulators. Such framework provides a common platform for assessing the performance of multimodal LZD algorithms globally. This work focuses on the first stage of the evaluation: Monte Carlo simulation-based assessment. Although straightforward, the absence of any common evaluation framework underscores the importance of this initial step in the right direction. The authors welcome

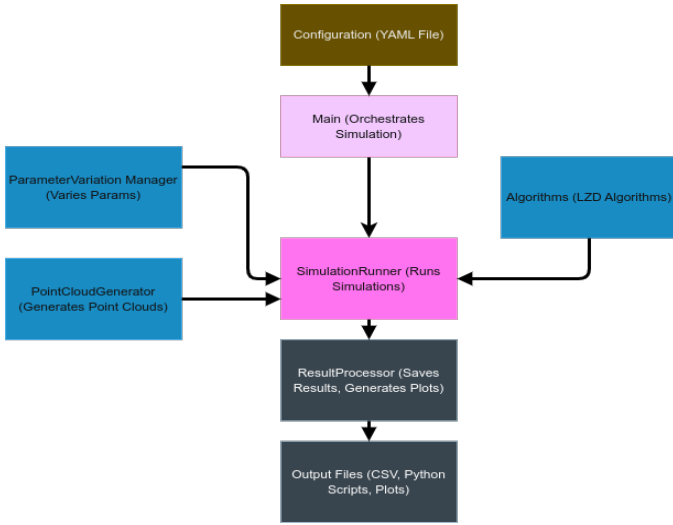


Fig. 9: High-level overview of the main software components for Monte Carlo simulations.

the development of a more comprehensive framework by the community.

The second stage evaluates LZDA performance across varying terrain complexities. Hyper-realistic simulation tools, such as IsaacSim and AirSim, facilitate the development of tailored algorithms for specific use cases, further promoting UAS service adoption.

The third stage shall address real-world and real-time challenges, including adverse environmental conditions, hardware limitations, and regulatory hurdles.

In this work only first and second stages were touched upon with a modest but meaningful step toward establishing a common evaluation framework for LZDAs.

B. Monte Carlo Simulation-Based Evaluation of Robustness, Precision, Accuracy, Computation Time, and Resources

To understand the Monte Carlo simulation-based evaluation framework, it is essential to examine its primary components: These components are illustrated in Figure 9.

- **Point Cloud Generator:** The Point Cloud Generator creates a point cloud with at most one feasible landing zone, if any exists. The point cloud consists of a central area (green) and a surrounding area (red), as shown in Figure 10. The surrounding area is characterized by high noise and unevenness, making it unlikely to contain a viable landing zone. The central area is customizable, allowing hazard metric values to be set to specific levels.
- **Parameter Variation Manager:** The Parameter Variation Manager generates hazard metric values, varying one metric at a time from an acceptable to an unacceptable range (or vice versa) while keeping other metrics within predefined thresholds.
- **Simulation Runner:** The Simulation Runner retrieves these parameters, passes them to the Point Cloud Generator, and invokes the Landing Zone Detection (LZD)

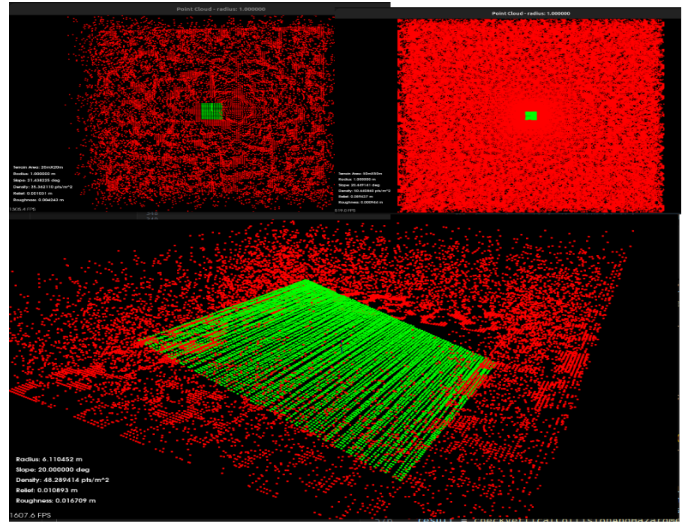


Fig. 10: Synthetically generated point clouds under controlled conditions. The top-left corner shows a 20mx20m area, while the top-right corner shows a 50mx50m area. The bottom part illustrates variations in point spacing, relief, and roughness visually.

algorithm on the generated point cloud for a preset number of iterations. This process is repeated for each hazard metric independently, adjusting its value across the acceptable-to-unacceptable range while maintaining other metrics within thresholds. Thresholds for hazard metrics, such as relief, roughness, slope, radius, and point density, are determined based on the UAS's landing gear ground clearance, maximum offset from surroundings, and intended operating height.

- **Result Processor:** Result Processor compiles the simulation outcomes and generates visualizations for analysis.

As shown in Figure 10, the point cloud area can be varied alongside hazard metric values. The top-left corner depicts a 20x20 meter area, while the top-right corner shows a 50x50 meter area. Investigating how point cloud area affects LZDA performance is of interest. The bottom part of Figure 10 illustrates synthetically generated variations in point spacing, relief, and roughness.

C. Monte Carlo Simulation-Based Evaluation Results Analysis

Figure 11 illustrates the varying success rates of the LZDAs. The region-growing segmentation-based LZDA demonstrates the highest success rate, while the inflating circles-based LZDA performs the worst. This poor performance is likely due to its random nature, large search area, and minimum iteration count of 10,000. Although none of the algorithms achieve exceptional performance, they adhere to the expected ideal pattern (dashed line) as can be seen in Figure 11. The grey dashed line represents the ideal performance expected from a perfect algorithm. Some algorithms perform better over other as expected highlighting their differences.

Success Rate (%) for All Algorithms Across Parameters (70m70m)

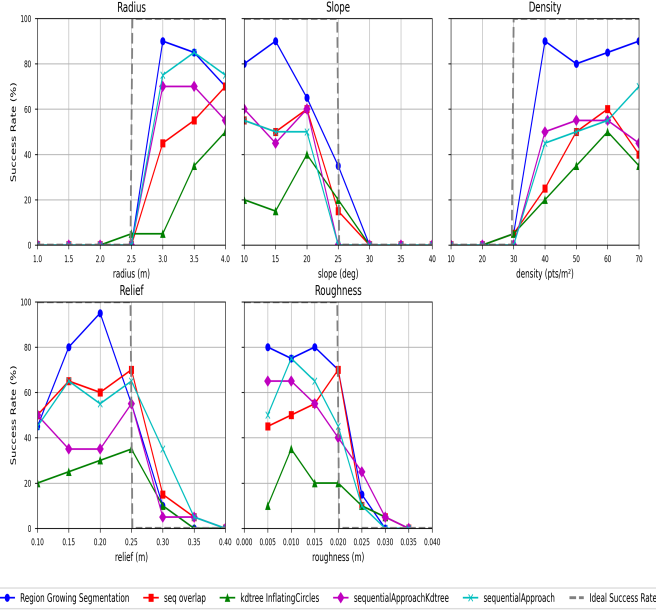


Fig. 11: Success rate plot for LZDAs implemented to find at least one landing zone over 20 iterations per x-value. The grey dashed line represents the ideal case plot.

Average CPU Time (s) for All Algorithms Across Parameters (70m70m)

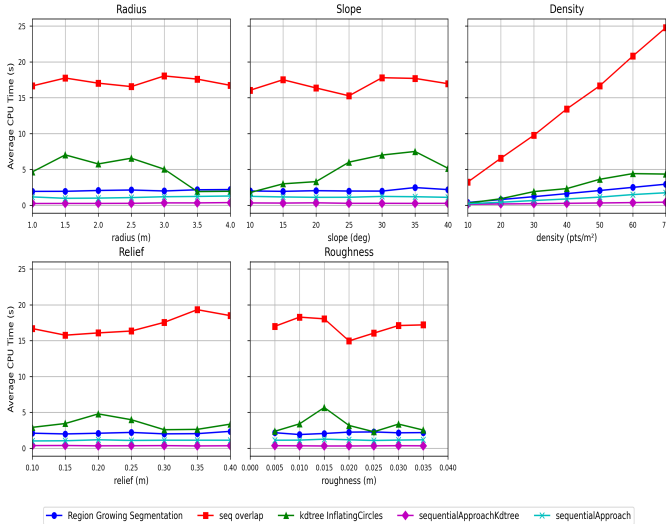


Fig. 12: Mean time plot for LZDAs implemented to find at least one landing zone over 20 iterations per x-value.

As shown in Figure 12, the sequential overlapping LZDA is the most time-consuming, whereas the customized sequential kd-tree-based LZDA is the fastest, leveraging the efficiency of the kd-tree data structure for point collection within window bounds. To the authors' knowledge, this approach represents a novel, albeit simple, improvement over existing methods. Working with this framework provided immediate feedback

Statistical Outlook of the Monte Carlo Simulation Results for Landing Zone Detection Algorithms (70m70m)

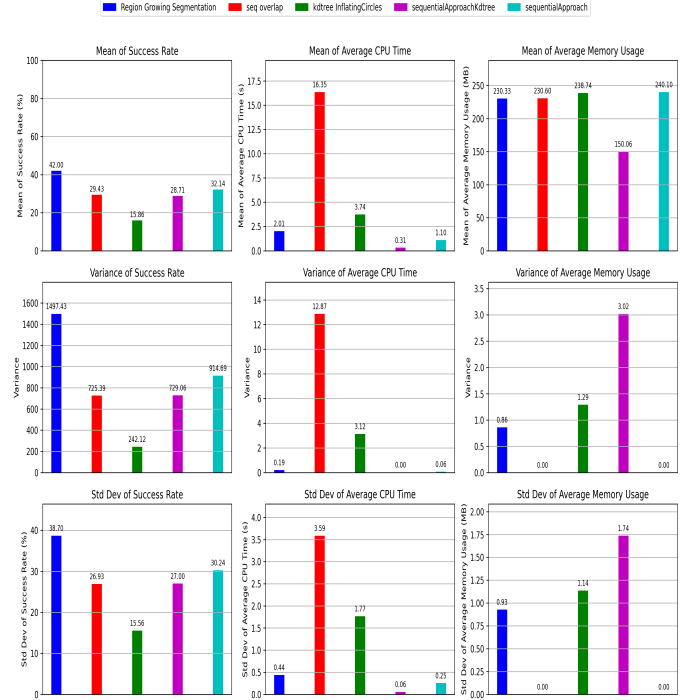


Fig. 13: Statistical overview of Monte Carlo simulations for LZDAs in a 70x70 meter area. The first row represents mean of success rate, time, and memory usage. The second row represents variance of success rate, time, and memory usage. The third row represents standard of deviation success rate, time, and memory usage. Each bar represents the result of 700 simulations (7 variations in each hazard metric value $\times$ 5 hazard metrics $\times$ 20 iterations per hazard metric value)

on new feature additions or algorithm modifications. Notably, the sequential overlapping LZDA's performance degrades significantly as point density increases, indicating the need for a downsampling filter to improve its efficiency. Mean memory usage for Region growing segmentation, sequential overlap, kd-tree Inflating circles and sequential approach methods is found 220-240MB, whereas the customized sequential kd-tree-based LZDA uses only 140-160MB of memory on a system configuration described in IV-B.

The statistical overview in Figure 13 highlights that the region-growing segmentation-based LZDA is the best choice for UAS applications prioritizing success rate. Conversely, for applications requiring minimal computation time and memory usage, the customized sequential kd-tree-based LZDA excels. Interestingly, while the kd-tree-based inflating circles LZDA has the lowest success rate, it is the most reliable, with the smallest standard deviation in success rate. In contrast, the region-growing segmentation-based LZDA is the least reliable, exhibiting the highest standard deviation. These results pertain to a 70x70 meter area, and performance may vary with different area sizes. This work demonstrates the utility of

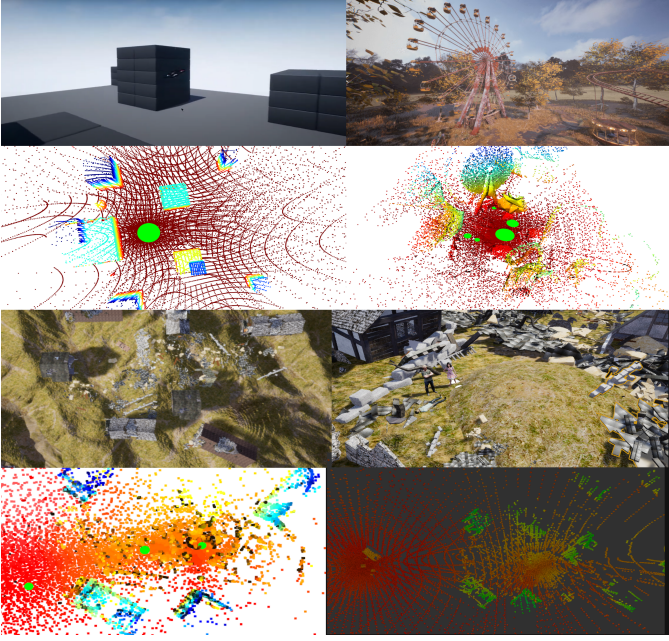


Fig. 14: AirSim environments used for testing LZDAs. The first row shows the simple environment and the moderate city environment. The second row shows the landing zones detected by the LZDAs. The third row shows a custom-built environment simulating a post-disaster scenario. The first image in the third row is a top view, while the second image is a close-up showing simulated humans and clutter. The first image in the last row shows the detected landing zones, and the final image shows an octomap generated from a simulated drone flight with a LiDAR in AirSim.

the Monte Carlo simulation-based evaluation framework for analyzing LZDA behavior. The framework provides a common platform for drone deployers, developers, and regulatory bodies to assess algorithm performance details. While this study focuses on point cloud-based simulations, the framework can be readily extended to real-world, multimodal datasets.

D. Step 2: Terrain Complexity Variation

Figure 14 depicts three distinct AirSim environments used to test the LZDAs. As can be seen in Figure 14 initial tests have successfully detected LZ in variety of terrain conditions. In future work point clouds generated in these environments will be taken as source of input for the Monte-Carlo simulation-based evaluation.

VI. CONCLUSION AND FUTURE WORK

A. Summary of Key Findings

During the development and evaluation of the LZDAs and the Monte Carlo-based evaluation in First stage, several significant findings emerged that merit discussion.

- The inability to achieve a 100 % success rate warrants further investigation. A likely cause is the failure to detect boundaries of flat patches. The current boundary

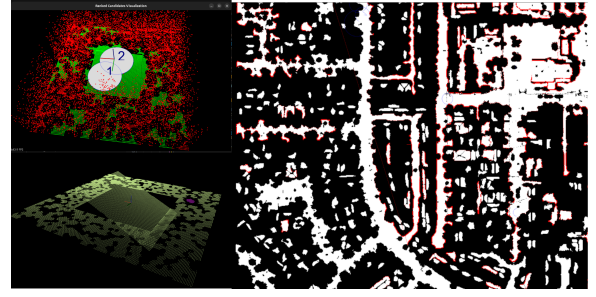


Fig. 15: Challenges in complex boundary segments extraction. The complex shapes of the region of interests to fit potential circular landing zones is shown. The red marking is the extracted shape result to fit circular LZ. Absence of red marking through the shape indicates failure to extract such complex boundary.

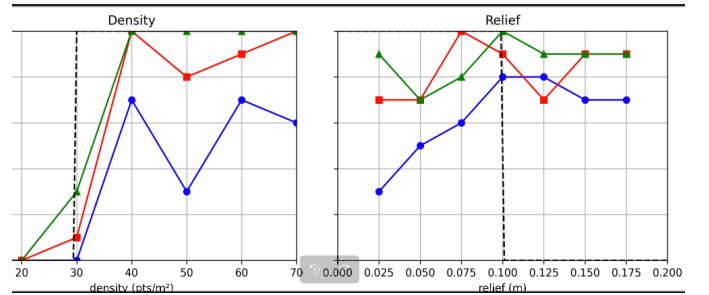


Fig. 16: Ideally the plots should be following the dashed line. Algorithms were finding solution where they were not supposed to find at all (left side of the dashed line no solutions shall be found) in relief related plot. Highlighting anomaly in relief calculation as feedback for correction.

shape estimation method lacks the robustness needed to accurately identify the boundaries of these flat patches.

- During the initial implementation of the LZDAs, errors were identified. The plots generated by the Monte Carlo-based evaluation framework provided critical feedback, highlighting implementation issues. For example, as shown in Figure 16, the relief plot immediately indicated the need for further investigation.
- The region-growing segmentation algorithm is highly sensitive, designed to detect even the slightest smooth, continuous flat surfaces. While this sensitivity enables the capture of flat surfaces, it also results in the identification of numerous small flat patches, as depicted in the bottom-left corner of Figure 15. The boundary extraction module struggles to process such complex shapes, contributing to failures in the LZDAs.
- The reported results were based on 20 simulations per value on the x-axis for a 70x70 meter area. Increasing the number of simulations is expected to improve result accuracy.

Based on the above findings the proposed framework can be used as a tool to tune heuristic rules for spe-

cific usecases along side highlighting anomalies in the implementation of the algorithms.

B. Future Research Directions

Future research will prioritize the following areas:

- Evaluating image and DL based Landing / Delivery Zone detection algorithms using this framework and expanding the framework's functionality as needed.
- Benchmark PX4 safe_landing_planner alongside image based LDZDAs.
- Providing a proof of concept using the framework's ability to optimizes and tunes algorithms for specific use cases simulated using IsaacSim to create dynamic, complex environments.
- Benchmarking the LZDAs on a UAS companion computer, such as the NVIDIA Jetson.
- Investigating UAS adoption requirements and regulatory compliance in greater depth.
- Developing a robust approach to handle the complex boundary extraction of flat patches. Alternatively, bypassing boundary extraction problem and using the inlier point cloud to formulate circular region fitting as an optimization problem, as implemented in [33] using Nelder-Mead algorithm.
- Deeper look into handling stage three challenges mentioned in V-A

REFERENCES

- [1] D. Mango, R. Opromolla, and C. Schmitt, "Hazard detection and landing site selection for planetary exploration using LIDAR," in *Proc. IEEE 7th Int. Workshop Metrol. Aerosp. (MetroAeroSpace)*, Pisa, Italy, Jun. 2020.
- [2] M. Brenner and P. Stütz, "Towards gesture-based cooperation with cargo handling unmanned aerial vehicles," *Int. J. Semantic Comput.*, vol. 17, pp. 411–434, 2023.
- [3] J. A. Subramanian, V. S. Asirvadani, S. A. B. M. Zulkifli, N. S. S. Singh, N. Shanthi, and R. K. Lagisetty, "Target localization for autonomous landing site detection: A review and preliminary result with static image photogrammetry," *Drones*, vol. 7, no. 8, p. 509, Aug. 2023. [Online]. Available: <https://www.mdpi.com/2504-446X/7/8/509>
- [4] Flytrex, "Delivery Process — Flytrex," *YouTube*, Nov. 20, 2022. [Online]. Available: <https://www.youtube.com/watch?v=IEjZwLoFsig>, Accessed: May 10, 2025, 0:20.
- [5] Federal Aviation Administration, "NEPA and Drones," *FAA.gov*, [Online]. Available: https://www.faa.gov/uas/advanced_operations/nepa_and_drones, Accessed: May 10, 2025.
- [6] M. D. Takahashi, A. Abershitz, R. Rubins, and M. S. Whalley, "Evaluation of safe landing area determination algorithms for autonomous rotorcraft using site benchmarking," *J. Amer. Helicopter Soc.*, vol. 58, no. 4, pp. 1–13, Oct. 2013.
- [7] G. Vitta Bukka and P. Stütz, "Evaluating point-cloud based safe landing zone detection algorithms for VTOL-UAV: An overview," in *Proc. Digit. Avionics Syst. Conf. (DASC)*, San Diego, CA, USA, 2024.
- [8] NVIDIA, "Isaac Sim - Robotics Simulation and Synthetic Data Generation," NVIDIA Developer, Apr. 2024. [Online]. Available: <https://developer.nvidia.com/isaacsim>, [Accessed: Jul. 06, 2025].
- [9] Microsoft, "AirSim: Open Source Simulator for Autonomous Vehicles," GitHub, Jul. 2022. [Online]. Available: <https://github.com/microsoft/AirSim>, [Accessed: Jul. 06, 2025].
- [10] J. Mackay, G. Ellingson, and T. W. McLain, "Landing zone determination for autonomous rotorcraft in surveillance applications," in *Proc. AIAA Guid., Navig., Control Conf.*, San Diego, CA, USA, Jan. 2016.
- [11] A. Massoud, A. Fahmy, U. Iqbal, S. Givigi, and A. Noureldin, "Real-time safe landing zone identification based on airborne LiDAR," *Sensors*, vol. 23, no. 7, p. 3491, Mar. 2023.
- [12] S. Scherer, L. Chamberlain, and S. Singh, "Autonomous landing at unprepared sites by a full-scale helicopter," *Robot. Auton. Syst.*, vol. 60, no. 12, pp. 1545–1562, Dec. 2012.
- [13] D. P. Singh and M. Yadav, "Suitable landing site selection for unmanned aerial vehicles using airborne laser scanning point cloud," *Int. Arch. Photogramm. Remote Sens. Spatial Inf. Sci.*, vol. 48, pp. 343–348, 2024.
- [14] N. A. M. C. da Veiga, "Safe landing site selection and autonomous landing of a UAV in unstructured environments," Lisbon, Portugal, 2022.
- [15] L. Yan, J. Qi, M. Wang, C. Wu, and J. Xin, "A safe landing site selection method of UAVs based on LiDAR point clouds," in *Proc. 39th Chin. Control Conf. (CCC)*, Shenyang, China, Jul. 2020.
- [16] R. B. Rusu and S. Cousins, "3D is here: Point Cloud Library (PCL)," in *Proc. IEEE Int. Conf. Robot. Autom. (ICRA)*, Shanghai, China, May 2011, pp. 1–4. [Online]. Available: <http://pointclouds.org>, Accessed: May 4, 2025.
- [17] CGAL Project, "CGAL, Computational Geometry Algorithms Library," [Online]. Available: <https://www.cgal.org>, Accessed: May 4, 2025.
- [18] Q.-Y. Zhou, J. Park, and V. Koltun, "Open3D: A modern library for 3D data processing," in *Proc. Int. Conf. 3D Vis. (3DV)*, Verona, Italy, Sep. 2018, pp. 1–8. [Online]. Available: <http://www.open3d.org>, Accessed: May 4, 2025.
- [19] T. Hinzmann, T. Stastny, C. Cadena, R. Siegwart, and I. Gilitschenski, "Free LSD: Prior-free visual landing site detection for autonomous planes," *IEEE Robot. Autom. Lett.*, vol. 3, no. 3, pp. 2545–2552, Jul. 2018.
- [20] L. Yang, C. Wang, and L. Wang, "Autonomous UAVs landing site selection from point cloud in unknown environments," *ISA Trans.*, vol. 130, pp. 610–628, Nov. 2022.
- [21] L. Yang and L. Wang, "An optimization-based selection approach of landing sites for swarm unmanned aerial vehicles in unknown environments," *Expert Syst. Appl.*, vol. 204, p. 117582, Oct. 2022.
- [22] A. E. Johnson, A. R. Klumpp, J. B. Collier, and A. A. Wolf, "Lidar-based hazard avoidance for safe landing on Mars," *J. Guid. Control Dyn.*, vol. 25, no. 6, pp. 1091–1099, Nov. 2002.
- [23] Y. Wang, J. Wang, W. Du, and N. Duan, "Point cloud plane fitting algorithm based on least square median," *Laser Optoelectron. Prog.*, vol. 60, no. 4, p. 0410019, Feb. 2023.
- [24] M. Mittal, R. Mohan, W. Burgard, and A. Valada, "Vision-based autonomous UAV navigation and landing for urban search and rescue," in *Proc. Int. Symp. Robot. Res. (ISRR)*, Hanoi, Vietnam, Oct. 2019.
- [25] N. Herath, O. De Silva, G. K. I. Mann, and A. Jayasiri, "AI-Driven Landing Zone Detection Module for Vertical Take-Off and Landing Vehicles using Projection-Based LiDAR-Navigation Pipelines," in *IEEE Trans. Autom. Sci. Eng.*, 2025.
- [26] C. R. Qi, H. Su, K. Mo, and L. J. Guibas, "PointNet: Deep Learning on Point Sets for 3D Classification and Segmentation," in *Proc. IEEE Conf. Comput. Vis. Pattern Recognit.*, Honolulu, HI, USA, 2017, pp. 652–660.
- [27] K. Zha and J. Yuan, "Landing Site Selection of Detector Based on 3D Point Cloud Segmentation," in *Proc. Int. Conf. Mech., Aerosp. Automat. Eng.*, Dec. 2021, pp. 104–112.
- [28] N. M. Balasooriya, O. De Silva, A. Jayasiri, and G. K. I. Mann, "AI-Based Landing Zone Detection for Vertical Takeoff and Land LiDAR Localization and Mapping Pipelines," in *Drone Syst. Appl.*, vol. 12, pp. 1–19, Jan. 2024.
- [29] PX4 Development Team, "PX4 avoidance ROS node for obstacle detection and avoidance," GitHub Repository, 2024. [Online]. Available: <https://github.com/PX4/PX4-Avoidance>
- [30] S. Sturges, "OpenLander: Free-to-use landing-zone detection for UAV using DNNs for object segmentation," GitHub Repository, 2024. [Online]. Available: <https://github.com/stephansturges/OpenLander>
- [31] OpenTopography, "OpenTopography: High-Resolution Topography Data and Tools," 2025. [Online]. Available: <https://opentopography.org>
- [32] M. Brenner, G. Vitta-Bukka, and P. Stütz, "Exploring multi-attribute decision making for collaborative first-aid drone deliveries," unpublished, 2024.
- [33] D. Nospes and P. Stütz, "Preflight aircraft emergency landing field estimation using optimization," in *2022 IEEE/AIAA 41st Digital Avionics Systems Conference (DASC)*, 2022, pp. 1–9, <https://doi.org/10.1109/DASC55683.2022.9925765>.